API Testing Basics

What is an API?

API stands for **Application Programming Interface**.

An API is a **bridge** or **messenger** that allows two software applications to communicate with each other.

Real-Time Example

Imagine you are in a restaurant.

- 👨 Customer → You
- 👨 Waiter → API
- 👨‍🍳 Kitchen → Server

Flow:

1. You place an order.
2. The waiter takes your order to the kitchen.
3. The kitchen prepares the food.
4. The waiter brings the food back to you.

Here, the **Waiter acts like an API**.

```
Customer  --->  Waiter(API)  --->  Kitchen(Server)
  Customer  <---  Waiter(API)  <---  Kitchen(Server)
```

The customer never directly talks to the kitchen.

Similarly,

- Frontend (Website/Mobile App)
- API
- Backend Server

communicate with each other.

Web/UI Testing vs API Testing vs Database Testing

Web/UI Testing	API Testing	Database Testing
Tests the user interface	Tests the APIs	Tests the database
Checks buttons, textboxes, links	Checks requests and responses	Checks stored data
Slower	Faster	Fast
Requires browser	No browser required	No browser required
Performed like a real user	Performed by sending HTTP requests	Performed using SQL queries

Example

Suppose you are logging into Amazon.

UI Testing

Checks:

- Login page opens
- Username textbox
- Password textbox
- Login button
- Error messages

API Testing

Checks:

- Login API is called
- Correct response is returned
- Authentication token is generated

Database Testing

Checks:

- User exists in database
- Password is stored correctly
- Login history updated

What is API Testing?

API Testing means verifying whether an API is working correctly.

Instead of clicking buttons on a webpage, we directly send requests to the server and verify the responses.

Example

Login API

```
POST /login
```

Request

```
Username = admin
  Password = admin123
```

Expected Response

```
{
      "status": "success",
      "token": "abcd12345"
  }
```

If the response is correct, the API test passes.

Why Do We Need API Testing?

API Testing is important because it helps us verify the backend logic without relying on the user interface.

Advantages

- Faster than UI testing
- Finds bugs early
- Tests business logic directly
- Works even if the UI is not developed
- Easy to automate
- More stable than UI tests
- Reduces testing time

What is Request?

A **Request** is the information sent from the client to the server asking it to perform an action.

Examples:

- Login
- Register
- Search product
- Place order
- Delete account

Example

```
Please give me product details.
```

This is a request.

What is Response?

A **Response** is the information returned by the server after processing the request.

Example

```
Product Name : Laptop
  Price : $800
  Stock : Available
```

This is the response.

Request and Response Flow

```
Client
     |
     | Request
     V
  Server
     |
     | Process Request
     V
  Database
     |
     | Response
     V
  Server
     |
     | Response
     V
  Client
```

Types of HTTP Requests

HTTP methods tell the server what action needs to be performed.

---

1. POST (Create)

Used to create a new resource.

Example

Create a new user.

```
POST /users
```

Request

```
{
      "name":"John",
      "age":25
  }
```

---

2. GET (Retrieve)

Used to retrieve existing data.

Example

```
GET /users/1
```

Response

```
{
      "id":1,
      "name":"John"
  }
```

---

3. PUT (Update)

Used to replace the entire resource.

Example

```
PUT /users/1
```

Request

```
{
      "name":"David",
      "age":30
  }
```

Entire user information gets updated.

---

4. PATCH (Partial Update)

Used to update only specific fields.

Example

```
PATCH /users/1
```

Request

```
{
      "age":31
  }
```

Only the age is updated.

---

5. DELETE

Used to delete a resource.

Example

```
DELETE /users/1
```

The user with ID 1 is deleted.

CRUD Operations

Operation	HTTP Method
Create	POST
Read	GET
Update	PUT
Partial Update	PATCH
Delete	DELETE

Remember:

```
Create  → POST
  Read    → GET
  Update  → PUT
  Modify   → PATCH
  Delete  → DELETE
```

Components of an API Response

A response usually contains the following parts:

```
Response

  ├── Status Code
  ├── Headers
  ├── Response Body
  ├── Response Time
  └── Cookies (Optional)
```

1. Status Code

Indicates whether the request was successful.

Example

```
200 OK
  404 Not Found
  500 Internal Server Error
```

---

2. Headers

Provide additional information about the response.

Example

```
Content-Type : application/json
```

---

3. Response Body

Contains the actual data returned by the server.

Example

```
{
      "id":1,
      "name":"John",
      "email":"john@test.com"
  }
```

---

4. Response Time

Shows how long the server took to respond.

Example

```
250 ms
```

---

5. Cookies (Optional)

Used to maintain user sessions.

What is Request Payload?

The **Request Payload** is the data sent from the client to the server.

Example

```
{
      "username":"admin",
      "password":"admin123"
  }
```

The server uses this data to process the request.

What is Response Payload?

The **Response Payload** is the data returned by the server.

Example

```
{
      "status":"success",
      "token":"ABC123XYZ"
  }
```

What Should We Test in an API Response?

During API testing, verify the following:

Status Code

Example

```
Expected : 200
  Actual   : 200
```

Response Time

Ensure the response is received within the expected time.

Example

```
Expected : Less than 2 seconds
```

Response Body

Verify:

- Correct values
- Required fields
- Data types
- Data format

Example

```
{
      "id":1,
      "name":"John"
  }
```

Response Headers

Verify headers such as:

```
Content-Type
  Authorization
  Cache-Control
```

Response Schema

Verify the JSON structure.

Example

```
{
      "id": Number,
      "name": String,
      "email": String
  }
```

Business Logic

Examples:

- User created successfully
- Product deleted successfully
- Invalid login returns proper error
- Duplicate email is rejected

Common HTTP Status Codes

2xx - Success

Code	Meaning
200	OK
201	Created
202	Accepted
204	No Content

3xx - Redirection

Code	Meaning
301	Moved Permanently
302	Found (Temporary Redirect)
304	Not Modified

4xx - Client Errors

These errors occur due to issues in the client's request.

Code	Meaning
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
409	Conflict
415	Unsupported Media Type
429	Too Many Requests

5xx - Server Errors

These errors occur due to issues on the server.

Code	Meaning
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout

Easy Way to Remember Status Codes

```
2xx → Success 😊

  3xx → Redirect 🔄

  4xx → Client Mistake 🙋

  5xx → Server Problem 💻
```

Summary

- API is a bridge between applications.
- API Testing verifies backend functionality without using the UI.
- A client sends a **Request** and receives a **Response**.
- HTTP methods perform CRUD operations:
  - POST → Create
  - GET → Retrieve
  - PUT → Update
  - PATCH → Partial Update
  - DELETE → Delete
- API responses include:
  - Status Code
  - Headers

    - Response Body
    - Response Time
    - Cookies (optional)
- Request Payload is the data sent to the server.
- Response Payload is the data returned by the server.
- Always verify status codes, headers, response body, response time, schema, and business logic during API testing.